# 데이터 생성 파이프라인 분석

## 1. 데이터 생성 파이프라인 개요

Open R1의 데이터 생성 파이프라인은 고품질 추론 트레이스를 대규모로 생성하는 시스템입니다. DeepSeek-R1과 같은 강력한 모델을 사용하여 수학, 코딩, 추론 문제에 대한 단계별 해결 과정을 생성합니다.

### 1.1 파이프라인 구조

```
입력 데이터셋 → vLLM 서버 → Distilabel 파이프라인 → 생성된 추론 트레이스 → HuggingFace
```

### 1.2 핵심 컴포넌트

- **Distilabel**: 대규모 데이터 생성을 위한 파이프라인 프레임워크
- **vLLM**: 고성능 추론 서버
- **Ray**: 분산 처리 및 병렬화
- **OpenAI API 호환**: 표준 API 인터페이스 사용

## 2. Distilabel 파이프라인 (`src/open_r1/generate.py`)

### 2.1 파이프라인 구축 함수

```
def build_distilabel_pipeline(
model: str,
```

```
base_url: str = "http://localhost:8000/v1",
prompt_column: Optional[str] = None,
prompt_template: str = "{{ instruction }}",
temperature: Optional[float] = None,
top_p: Optional[float] = None,
max_new_tokens: int = 8192,
num_generations: int = 1,
input_batch_size: int = 64,
client_replicas: int = 1,
timeout: int = 900,
retries: int = 0,
) -> Pipeline:
"""Distilabel 파이프라인 구축"""
# 1. 생성 매개변수 설정
generation_kwargs = {"max_new_tokens": max_new_tokens}
if temperature is not None:
generation_kwargs["temperature"] = temperature
if top_p is not None:
generation_kwargs["top_p"] = top_p
# 2. Ray 기반 파이프라인 생성
with Pipeline().ray() as pipeline:
TextGeneration(
llm=OpenAILLM(
```

```
base_url=base_url,
api_key="something", # vLLM은 API 키 불필요
model=model,
timeout=timeout,
max_retries=retries,
generation_kwargs=generation_kwargs,
),
template=prompt_template,
input_mappings={"instruction": prompt_column} if prompt_column else {},
input_batch_size=input_batch_size,
num_generations=num_generations,
group_generations=True, # 여러 생성 결과를 그룹화
resources=StepResources(replicas=client_replicas),
)
return pipeline
```

## 2.2 주요 매개변수 설명

| 매개변수 | 기본값 | 설명 |
|---|---|---|
| `model` | - | 사용할 모델명 (vLLM 서버에 로드된 모델) |
| `base_url` | `localhost:8000/v1` | vLLM 서버 URL |
| `prompt_template` | `{{ instruction }}` | 프롬프트 템플릿 (Jinja2 형식) |

| max_new_tokens | 8192 | 최대 생성 토큰 수 |

| num_generations | 1 | 문제당 생성할 응답 수 |

| input_batch_size | 64 | 입력 배치 크기 |

| client_replicas | 1 | 클라이언트 복제본 수 |

| timeout | 900 | 요청 타임아웃 (초) |

## 2.3 템플릿 시스템

### 기본 템플릿

```
{{ instruction }}
```

### 수학 문제용 템플릿

```
You will be given a problem. Please reason step by step, and put your final answ

{{ prompt }}
```

### 코딩 문제용 템플릿

```
Solve the following programming problem step by step:

{{ problem_statement }}

Please provide your solution with clear reasoning and well-commented code.
```

## 3. 고성능 추론 생성 (scripts/generate_reasoning.py)

### 3.1 비동기 생성 시스템

```
async def generate_completion(session, prompt, args):
"""단일 완성 생성"""
retry_budget = 10
while retry_budget > 0:
try:
await asyncio.sleep(random.uniform(0.0, 0.1)) # 지터 추가
async with session.post(
f"http://{args.api_addr}/v1/chat/completions",
json={
"model": "default",
"messages": [{"role": "user", "content": prompt}],
"max_tokens": args.max_tokens,
"temperature": args.temperature,
"top_p": args.top_p,
},
headers={"Authorization": "Bearer EMPTY"},
) as response:
return await response.json(content_type=None)
except Exception as e:
```

```
print(f"API error (will retry): {e}")
retry_budget -= 1
await asyncio.sleep(10) # 재시도 전 대기
return None
```

### 3.2 배치 처리 및 동시성 제어

```
async def process_example(example, session, args, output_file, pbar):
"""단일 예시 처리"""
prompt = args.prompt_template.format(prompt=example[args.prompt_column])
try:
# 여러 생성 작업을 병렬로 실행
tasks = [generate_completion(session, prompt, args)
for _ in range(args.num_generations)]
completions = await asyncio.gather(*tasks)
if any(completion is None for completion in completions):
print(f"Error processing example")
return None
# 결과 추출
generations = []
finish_reasons = []
api_metadata = []
for completion in completions:
```

```
generations.append(completion["choices"][0]["message"]["content"])

finish_reasons.append(completion["choices"][0]["finish_reason"])

api_metadata.append(completion["usage"])

# 원본 데이터와 생성 결과 결합

result = {

**example, # 원본 필드 보존

"generations": generations,

"finish_reasons": finish_reasons,

"api_metadata": api_metadata,

}

# 파일에 안전하게 쓰기

async with file_lock:

async with aiofiles.open(output_file, mode="a") as f:

await f.write(json.dumps(result) + "\n")

await f.flush()

return result

except Exception as e:

print(f"Error processing example: {e}")

return None
```

### 3.3 동시성 관리

```
async def main():
```

```
# 동시 작업 집합
active_tasks: Set[asyncio.Task] = set()
# 진행률 표시
pbar = tqdm(
total=len(dataset) - len(processed_uuids),
desc="Generating responses",
unit="row",
mininterval=2,
smoothing=0.0001,
)
async  with aiohttp.ClientSession(
timeout=aiohttp.ClientTimeout(total=60 * 60),
connector=aiohttp.TCPConnector(
limit=args.max_concurrent,
ttl_dns_cache=300,
keepalive_timeout=60 * 60
),
) as session:
for example in dataset:
# 동시성 제한 확인
while  len(active_tasks) >= args.max_concurrent:
done, active_tasks = await asyncio.wait(
active_tasks,
```

```
return_when=asyncio.FIRST_COMPLETED
)
for task in done:
try:
await task
except Exception as e:
print(f"Task failed: {e}")
# 새 작업 생성
task = asyncio.create_task(
process_example(example, session, args, args.output_file, pbar)
)
active_tasks.add(task)
task.add_done_callback(active_tasks.discard)
# 남은 작업 완료 대기
if active_tasks:
await asyncio.gather(*active_tasks, return_exceptions=True)
```

# 4. 데이터 생성 워크플로우

## 4.1 단계별 프로세스

### 1단계: vLLM 서버 시작

```
# DeepSeek-R1 모델로 vLLM 서버 시작
```

```
vllm serve deepseek-ai/DeepSeek-R1-Distill-Qwen-7B \
--host 0.0.0.0 \
--port 8000 \
--tensor-parallel-size 4 \
--max-model-len 32768 \
--trust-remote-code
```

### 2단계: 데이터셋 준비

```
from datasets import load_dataset

# 기존 데이터셋 로드
dataset = load_dataset("hendrycks/competition_math", split="train")

# 또는 커스텀 데이터셋
dataset = load_dataset("your-org/your-dataset", split="train")
```

### 3단계: Distilabel 파이프라인 실행

```
python src/open_r1/generate.py \
--hf-dataset hendrycks/competition_math \
--hf-dataset-split train \
--prompt-column problem \
--prompt-template "Solve this step by step:\n{{ instruction }}" \
```

```
--model deepseek-ai/DeepSeek-R1-Distill-Qwen-7B \
--vllm-server-url  http://localhost:8000/v1 \
--temperature 0.7 \
--top-p  0.9 \
--max-new-tokens 4096 \
--num-generations  4 \
--input-batch-size 32 \
--client-replicas  4 \
--hf-output-dataset your-org/generated-reasoning-traces
```

**4단계: 고성능 생성 (대안)**

```
python  scripts/generate_reasoning.py \
--dataset-name hendrycks/competition_math \
--output-file  reasoning_traces.jsonl \
--prompt-column problem \
--uuid-column  problem \
--api-addr localhost:8000 \
--num-generations  4 \
--temperature 0.6 \
--top-p  0.95 \
--max-tokens 16384 \
--max-concurrent  100
```

## 4.2 생성 결과 형식

### Distilabel 출력 형식

```
{
"instruction": "원본 문제",
"generations": [
"<think>\n단계별 추론...\n</think>\n<answer>\n최종 답변\n</answer>",
"<think>\n다른 접근법...\n</think>\n<answer>\n답변\n</answer>"
],
"model_name": "deepseek-ai/DeepSeek-R1-Distill-Qwen-7B",
"distilabel_metadata": {
"raw_output_text_generation": "...",
"statistics_text_generation": {...}
}
}
```

### 고성능 생성 출력 형식

```
{
"problem": "원본 문제",
"solution": "원본 해답",
"generations": [
"생성된 추론 트레이스 1",
"생성된 추론 트레이스 2",
```

```
"생성된 추론 트레이스 3",
"생성된 추론 트레이스 4"
],
"finish_reasons": ["stop", "stop", "stop", "stop"],
"api_metadata": [
{"prompt_tokens": 150, "completion_tokens": 800, "total_tokens": 950},
{"prompt_tokens": 150, "completion_tokens": 750, "total_tokens": 900},
{"prompt_tokens": 150, "completion_tokens": 900, "total_tokens": 1050},
{"prompt_tokens": 150, "completion_tokens": 650, "total_tokens": 800}
]
}
```


# 5. 성능 최적화 전략

## 5.1 vLLM 서버 최적화

### 텐서 병렬화

```
# 4개 GPU에 모델 분산
--tensor-parallel-size 4

# 파이프라인 병렬화 (대형 모델용)
--pipeline-parallel-size 2
```

### 메모리 최적화

```
# KV 캐시 최적화
--kv-cache-dtype  fp8

# 최대 배치 크기 조정
--max-num-batched-tokens  8192

# 최대 시퀀스 길이 설정
--max-model-len  32768
```

### 처리량 최적화

```
# 연속 배치 처리
--enable-chunked-prefill

# 투기적 디코딩
--speculative-model  small-model-name

# GPU 메모리 활용률 조정
--gpu-memory-utilization  0.9
```

## 5.2 클라이언트 최적화

### 동시성 튜닝

```
# 최적 동시성 수준 찾기
max_concurrent = min(
vllm_server_capacity * 2, # 서버 용량의 2배
available_memory // request_size, # 메모리 제한
1000  # 상한선
)
```

### 배치 크기 최적화

```
# GPU 메모리에 따른 배치 크기 조정
if gpu_memory_gb >= 80:
input_batch_size = 128
elif gpu_memory_gb >= 40:
input_batch_size = 64
else:
input_batch_size = 32
```

### 재시도 전략

```
# 지수 백오프 재시도
async  def  retry_with_backoff(func, max_retries=5):
for attempt in  range(max_retries):
try:
```

```
return  await func()

except Exception as e:

if attempt == max_retries - 1:

raise e

wait_time = (2 ** attempt) + random.uniform(0, 1)

await asyncio.sleep(wait_time)
```

## 5.3 분산 처리

### Ray 클러스터 설정

```
# Ray 클러스터 초기화

import ray

ray.init(address="ray://head-node:10001")


# 분산 파이프라인 실행

with Pipeline().ray() as pipeline:

# 여러 노드에 작업 분산

TextGeneration(

resources=StepResources(

replicas=8, # 8개 복제본

cpus=2, # 복제본당 2 CPU

gpus=0.5  # 복제본당 0.5 GPU

)
```

```
)
```

**멀티노드 생성**

```
# 헤드 노드
ray start --head --port=6379

# 워커 노드들
ray start --address=head-node:6379

# 분산 생성 실행
python src/open_r1/generate.py \
--client-replicas 16 \
--input-batch-size 128
```

# 6. 품질 관리 및 필터링

## 6.1 생성 품질 검증

```
def validate_generation(generation: str) -> bool:
"""생성된 추론 트레이스 품질 검증"""
# 1. 형식 검증
if not re.match(r"<think>.*</think>.*<answer>.*</answer>",
generation, re.DOTALL):
```

```
return False

# 2. 최소 길이 검증

if  len(generation) < 100:

return False

# 3. 추론 단계 검증

think_content = re.search(r"<think>(.*?)</think>",

generation, re.DOTALL)

if think_content:

reasoning = think_content.group(1)

# 단계별 추론 패턴 확인

step_patterns = [

r"Step \d+:",

r"\d+\.",

r"First,|Next,|Then,|Finally,"

]

if not any(re.search(pattern, reasoning) for pattern in step_patterns):

return False

# 4. 답변 형식 검증

answer_content = re.search(r"<answer>(.*?)</answer>",

generation, re.DOTALL)

if not answer_content or len(answer_content.group(1).strip()) < 10:

return False
```

```
return True
```

## 6.2 중복 제거

```
def deduplicate_generations(generations: list[str]) -> list[str]:
"""생성된 추론 트레이스 중복 제거"""
# 1. 정확한 중복 제거
unique_generations = list(set(generations))
# 2. 의미적 유사성 기반 중복 제거
from sentence_transformers import SentenceTransformer
model = SentenceTransformer('all-MiniLM-L6-v2')
embeddings = model.encode(unique_generations)
# 코사인 유사도 계산
from sklearn.metrics.pairwise import cosine_similarity
similarity_matrix = cosine_similarity(embeddings)
# 높은 유사도 (>0.9) 항목 제거
to_remove = set()
for i in range(len(similarity_matrix)):
for j in range(i+1, len(similarity_matrix)):
if similarity_matrix[i][j] > 0.9:
to_remove.add(j)
filtered_generations = [gen for i, gen in enumerate(unique_generations)
if i not in to_remove]
```

```
return filtered_generations
```

### 6.3 품질 점수 계산

```
def  calculate_quality_score(generation: str, problem: str) -> float:
"""생성 품질 점수 계산"""
score = 0.0
# 1. 형식 점수 (0-0.3)
if validate_generation(generation):
score += 0.3
# 2. 추론 깊이 점수 (0-0.3)
reasoning_steps = count_reasoning_steps(generation)
score += min(0.3, reasoning_steps * 0.1)
# 3. 관련성 점수 (0-0.2)
relevance = calculate_relevance(generation, problem)
score += relevance * 0.2
# 4. 다양성 점수 (0-0.2)
diversity = calculate_diversity(generation)
score += diversity * 0.2
return  min(1.0, score)
```

## 7. 실제 사용 사례

## 7.1 Mixture-of-Thoughts 데이터셋 생성

```
# 1. 수학 문제 추론 트레이스 생성
python src/open_r1/generate.py \
--hf-dataset hendrycks/competition_math \
--prompt-template "Solve step by step and put your final answer in \\boxed{}:\n
--model deepseek-ai/DeepSeek-R1-Distill-Qwen-7B \
--num-generations 4 \
--temperature 0.7 \
--hf-output-dataset open-r1/math-reasoning-traces

# 2. 코딩 문제 추론 트레이스 생성
python src/open_r1/generate.py \
--hf-dataset openai/humaneval \
--prompt-template "Solve this programming problem step by step:\n{{ prompt }}"
--model deepseek-ai/DeepSeek-R1-Distill-Qwen-7B \
--num-generations 4 \
--temperature 0.6 \
--hf-output-dataset open-r1/code-reasoning-traces

# 3. 과학 문제 추론 트레이스 생성
python src/open_r1/generate.py \
--hf-dataset allenai/ai2_arc \
```

```
--prompt-template "Think through this science question step by step:\n{{ questi
--model deepseek-ai/DeepSeek-R1-Distill-Qwen-7B \
--num-generations 4 \
--temperature 0.8 \
--hf-output-dataset open-r1/science-reasoning-traces
```

### 7.2 대규모 병렬 생성

```
# 멀티 GPU 클러스터에서 대규모 생성
sbatch --nodes=4 --gres=gpu:8 << 'EOF'
#!/bin/bash

# 각 노드에서 vLLM 서버 시작
srun --ntasks-per-node=1 vllm serve \
deepseek-ai/DeepSeek-R1-Distill-Qwen-7B \
--tensor-parallel-size 8 \
--host 0.0.0.0 \
--port 8000 &

sleep 60 # 서버 시작 대기

# 분산 생성 실행
python src/open_r1/generate.py \
```

```
--hf-dataset your-large-dataset \

--client-replicas 32 \

--input-batch-size 256 \

--num-generations 8 \

--hf-output-dataset your-output-dataset



EOF
```

### 7.3 품질 필터링 파이프라인

```
# 생성 후 품질 필터링

from datasets import load_dataset



# 생성된 데이터셋 로드

dataset = load_dataset("your-org/generated-traces")



# 품질 필터링 적용

def  filter_high_quality(example):

generations = example["generations"]

# 각 생성에 대해 품질 점수 계산

quality_scores = [

calculate_quality_score(gen, example["problem"])

for gen in generations
```

```
]

# 고품질 생성만 유지 (점수 > 0.7)

high_quality_gens = [

gen for gen, score in  zip(generations, quality_scores)

if score > 0.7

]

if  len(high_quality_gens) > 0:

example["generations"] = high_quality_gens

return example

else:

return None



# 필터링 적용

filtered_dataset = dataset.map(

filter_high_quality,

remove_columns=["distilabel_metadata"]

).filter(lambda  x: x is  not None)



# 필터링된 데이터셋 저장

filtered_dataset.push_to_hub("your-org/high-quality-traces")
```

## 8. 모니터링 및 디버깅

### 8.1 생성 통계 추적

```
import wandb

# W&B 초기화
wandb.init(project="open-r1-data-generation")

# 생성 통계 로깅
def  log_generation_stats(generations, problems):
stats = {
"total_generations": len(generations),
"avg_length": np.mean([len(gen) for gen in generations]),
"format_compliance": sum(validate_generation(gen) for gen in generations) / len(
"avg_reasoning_steps": np.mean([count_reasoning_steps(gen) for gen in generation
}
wandb.log(stats)
```

### 8.2 오류 모니터링

```
import logging

# 로깅 설정
logging.basicConfig(
```

```
level=logging.INFO,
format='%(asctime)s - %(levelname)s - %(message)s',
handlers=[
logging.FileHandler('generation.log'),
logging.StreamHandler()
]
)


# 오류 추적
def  track_errors(func):
def  wrapper(*args, **kwargs):
try:
return func(*args, **kwargs)
except Exception as e:
logging.error(f"Error in {func.__name__}: {e}")
# 오류 통계 업데이트
wandb.log({"errors": 1})
raise
return wrapper
```

## 8.3 성능 모니터링

```
import time
```

```
import psutil


class PerformanceMonitor:

def  __init__(self):

self.start_time = time.time()

self.processed_count = 0

def  update(self, batch_size):

self.processed_count += batch_size

# 처리량 계산

elapsed = time.time() - self.start_time

throughput = self.processed_count / elapsed

# 시스템 리소스 모니터링

cpu_percent = psutil.cpu_percent()

memory_percent = psutil.virtual_memory().percent

# 로깅

wandb.log({

"throughput": throughput,

"cpu_usage": cpu_percent,

"memory_usage": memory_percent,

"processed_total": self.processed_count

})
```

데이터 생성 파이프라인은 Open R1의 핵심 구성 요소로, 고품질 추론 트레이스를 대규모로 생성하여 모델 훈련의 기반을 제공합니다. 효율적인 병렬 처리와 품질 관리를 통해 DeepSeek-R1 수준의 추론 능력을 학습할 수 있는 데이터를 생성합니다.